

OTA SWU 서명 알고리즘 선택 절차서 (RSA-3072 / ML-DSA-87)

1. 개요

항목	내용
대상	SWUpdate OTA 패키지(.swu)의 CMS 서명과, 디바이스가 검증에 쓰는 신뢰 인증서
브랜치	shiba-meta-secure-boot ota-mldsa87 (base: release-toe2)
서명 대상	.swu 안의 sw-description (CMS, 서명 속성 다이제스트 SHA-384)
지원 알고리즘	RSA-3072, ML-DSA-87
실기 검증	2026-10-07, grey 보드에서 rsa3072 → dual → mldsa87 OTA, mldsa87 장비가 RSA 서명 .swu를 거부함

빌드할 때 두 가지를 정한다.

- 디바이스가 신뢰할 인증서 (SWU_VERIFY_MODE): 이미지 rootfs의 /usr/share/swupdate/swu-sign-cert.pem에 들어간다. 이 이미지가 설치된 뒤 다음 OTA부터 적용된다.
- 이번 .swu를 서명할 알고리즘 (SWU_SIGN_ALG): 지금 디바이스에 설치돼 있는 이미지가 신뢰하는 알고리즘이어야 한다.

2. 모드와 결과물

SWU_VERIFY_MODE	SWU_SIGN_ALG	디바이스 신뢰 인증서	.swu 서명	swupdate OpenSSL 설정
rsa3072 (기본값)	rsa3072 (고정)	RSA-3072 1개	RSA-3072	/etc/ssl/openssl.cnf (FIPS 전용, 기존과 동일)
dual	rsa3072 (기본값)	RSA-3072 + ML-DSA-87	RSA-3072	/etc/ssl/swupdate.cnf
dual	mldsa87	RSA-3072 + ML-DSA-87	ML-DSA-87	/etc/ssl/swupdate.cnf
mldsa87	mldsa87 (고정)	ML-DSA-87 1개	ML-DSA-87	/etc/ssl/swupdate.cnf

- rsa3072·mldsa87 모드에서 서명 알고리즘을 반대로 지정하면 빌드가 파싱 단계에서 멈춘다.
- dual에서는 SWU_SIGN_ALG를 바꿔도 이미지 내용(인증서, 설정 파일)은 같고 .swu 서명만 바뀐다.
- dual·mldsa87 이미지는 swupdate 전용 OpenSSL 설정 /etc/ssl/swupdate.cnf와 /etc/swupdate/conf.d/10-openssl을 함께 설치한다. FIPS 3.1.2 provider에는 ML-DSA가 없어서 default provider를 쓰므로, 이 두 모드에서는 swupdate의 서명 검증이 FIPS 경계 밖이다. 다른 프로그램은 계속 /etc/ssl/openssl.cnf (FIPS 전용)를 쓴다.

3. 서명 키·인증서 준비

3.1 위치

빌드는 키를 만들지 않는다. 빌드 머신의 SIGN_KEY_PATH(기본 ~/STM32AP_KeyGen) 아래에 미리 넣어 둔다. 원본은 /opt/STM32AP_KeyGen에 보관한다.

```

${SIGN_KEY_PATH}/
├── swu-sign-3072/          # RSA-3072
│   ├── swu-sign-key.pem   # 개인키 (0600, 암호 없음)
│   └── swu-sign-cert.pem  # 자체 서명 인증서
└── swu-sign-mldsa87/      # ML-DSA-87
    ├── swu-sign-key.pem
    └── swu-sign-cert.pem

```

모드	필요한 파일
rsa3072	swu-sign-3072/ 키 + 인증서
mldsa87	swu-sign-mldsa87/ 키 + 인증서
dual	두 인증서 모두 + SWU_SIGN_ALG에 해당하는 키

- 디바이스는 인증서 파일 그 자체를 신뢰한다. 빌드 머신이 여러 대면 같은 키·인증서를 복사해 써야 한다. 머신마다 새로 만들면 다른 머신에서 만든 .swu를 디바이스가 거부한다.
- 개인키는 암호 없이 저장한다. 빌드의 서명 명령(`openssl cms -sign -inkey ...`)이 암호를 넘기지 않는다.

3.2 ML-DSA-87 키 생성

OpenSSL 3.5 이상이 필요하다. orb(Ubuntu 22.04)의 시스템 openssl은 3.0.2라 쓸 수 없으므로, 한 번 빌드한 트리의 openssl-native(3.5.4)를 쓴다.

```

cd /home/yslim/yocto/tpm-spi/build-openstlinuxweston-shiba-ota-grey
O=$(ls -d tmp-glibc/sysroots-components/*/openssl-native/usr | head -1)
export LD_LIBRARY_PATH=$PWD/$O/lib
OPENSSL=$PWD/$O/bin/openssl
$OPENSSL version                # 3.5 이상인지 확인

K=~/.STM32AP_KeyGen/swu-sign-mldsa87
mkdir -p $K && cd $K
(umask 077; $OPENSSL genpkey -algorithm ML-DSA-87 -out swu-sign-key.pem)
$OPENSSL req -new -x509 -config /dev/null -key swu-sign-key.pem -out swu-sign-cert.pem \
    -days 7300 -subj "/CN=shiba SWU Key ML-DSA-87"

```

Mac에서는 Homebrew OpenSSL(/opt/homebrew/bin/openssl, 3.6.x)로 같은 명령을 쓸 수 있다. /usr/bin/openssl은 LibreSSL이라 ML-DSA를 지원하지 않는다.

3.3 RSA-3072 키 생성 (새로 만들 때만)

기존 swu-sign-3072/를 계속 쓰면 이 절은 건너뛴다. 키를 바꾸면 기존 디바이스가 새 .swu를 받지 못한다.

```

K=~/.STM32AP_KeyGen/swu-sign-3072
mkdir -p $K && cd $K
(umask 077; openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out swu-sign-key.pem)

```

```
openssl req -new -x509 -config /dev/null -key swu-sign-key.pem -out swu-sign-cert.pem \
    -sha384 -days 7300 -subj "/CN=shiba SWU Key"
```

3.4 확인

```
cd ~/STM32AP_KeyGen
$OPENSSL x509 -in swu-sign-3072/swu-sign-cert.pem -noout -subject -enddate -text | grep
-E 'subject|Not After|Public Key Algorithm|Public-Key'
$OPENSSL x509 -in swu-sign-mldsa87/swu-sign-cert.pem -noout -subject -enddate -text | grep
-E 'subject|Not After|Public Key Algorithm'
# 키와 인증서가 짝인지 (두 해시가 같아야 함)
d=swu-sign-mldsa87
$OPENSSL pkey -in $d/swu-sign-key.pem -pubout -outform DER | $OPENSSL dgst -sha256
$OPENSSL x509 -in $d/swu-sign-cert.pem -pubkey -noout | $OPENSSL pkey -pubin -pubout
-outform DER | $OPENSSL dgst -sha256
```

인증서	기대값
RSA	Public Key Algorithm: rsaEncryption, Public-Key: (3072 bit)
ML-DSA	Public Key Algorithm: ML-DSA-87

빌드도 같은 검사를 하며, 맞지 않으면 빌드가 멈춘다.

4. 빌드

4.1 local.conf 설정

빌드 디렉터리의 `conf/local.conf` 끝에 원하는 조합을 넣는다. 아무것도 넣지 않으면 `rsa3072` 이다.

```
# RSA-3072만 신뢰, RSA 서명 (기본값과 같음)
SWU_VERIFY_MODE = "rsa3072"

# 둘 다 신뢰, RSA 서명
SWU_VERIFY_MODE = "dual"

# 둘 다 신뢰, ML-DSA 서명
SWU_VERIFY_MODE = "dual"
SWU_SIGN_ALG = "mldsa87"

# ML-DSA-87만 신뢰, ML-DSA 서명
SWU_VERIFY_MODE = "mldsa87"
```

`SIGN_KEY_PATH`를 기본값(`~/STM32AP_KeyGen`)이 아닌 곳으로 쓰려면 같은 파일에 `SIGN_KEY_PATH = "/경로"`를 넣는다.

4.2 적용 값 확인

```
cd /home/yslim/yocto/tpm-spi
source layers/openembedded-core/oe-init-build-env build-openstlinuxweston-shiba-ota-grey
bitbake-getvar --value SWU_VERIFY_MODE -r swu-keygen
bitbake-getvar --value SWU_SIGN_ALG -r swu-keygen
bitbake-getvar --value SWU_VERIFY_CERTS -r swu-keygen # 디바이스에 들어갈 인증서 목록 (RSA 먼저)
bitbake-getvar --value SWU_SIGN_KEY_DIR -r swu-image-secure
```

4.3 빌드 명령

```
bitbake st-image-secure      # scripts/run bitbake
bitbake swu-image-secure     # scripts/run gen-swu (cleansstate 후 빌드)
```

- 신뢰 인증서는 swu-keygen 패키지가 rootfs에 넣는다. 이 레시피는 매번 다시 실행(nostamp)되므로 키 디렉터리 내용이 바뀌어도 반영된다.
- .swu 서명은 swu-image-secure가 한다.
- 결과물: tmp-glibc/deploy/images/<MACHINE>/swupdate/swu-image-secure-<MACHINE>-<version>.swu

4.4 결과물 서명 확인

.swu 파일 이름에는 서명 알고리즘이 들어가지 않고, 버전이 같으면 같은 이름으로 덮어쓴다. 배포 전에 서명을 확인하고, 보관할 때는 이름에 알고리즘을 붙여 두는 것이 좋다 (예: ...-r160-20261007-rsa3072.swu).

```
mkdir x && cd x && cpio -idv < ../swu-image-secure-<MACHINE>-<version>.swu
openssl cms -cmsout -print -inform DER -in sw-description.sig | grep -A1 -E
'signatureAlgorithm:|subject:'
```

서명	signatureAlgorithm	sw-description.sig 크기
RSA-3072	rsaEncryption	약 2 KB
ML-DSA-87	ML-DSA-87 (2.16.840.1.101.3.4.3.19)	약 12 KB

인증서로 실제 검증까지 해 보려면:

```
openssl cms -verify -inform DER -in sw-description.sig -content sw-description -binary \
  -CAfile ~/STM32AP_KeyGen/swu-sign-mldsa87/swu-sign-cert.pem -purpose any -out /dev/null
# CMS Verification successful
```

여기서 openssl은 3.5 이상이어야 한다 (3.2절).

5. 서명 알고리즘 전환 절차

5.1 규칙

.swu는 지금 디바이스에 설치된 이미지가 신뢰하는 알고리즘으로 서명한다. 새로 빌드하는 이미지의 모드는 설치가 끝난 뒤부터 적용된다.

현재 디바이스	받을 수 있는 서명	설치할 수 있는 이미지 모드
rsa3072	RSA만	rsa3072, dual (RSA 서명)
dual	RSA, ML-DSA	모든 모드 (해당 모드가 허용하는 서명)
mldsa87	ML-DSA만	mldsa87, dual (ML-DSA 서명)

그래서 rsa3072 ? mldsa87 사이는 한 번에 갈 수 없고 항상 dual을 거친다.

5.2 RSA-3072 → ML-DSA-87

단계	local.conf	.swu 서명	설치 후 디바이스
1	SWU_VERIFY_MODE = "dual"	RSA (기본값)	dual (RSA + ML-DSA 신뢰)
2 (선택)	SWU_VERIFY_MODE = "dual", SWU_SIGN_ALG = "mldsa87"	ML-DSA	dual. ML-DSA 서명이 통과하는지 미리 확인하는 단계
3	SWU_VERIFY_MODE = "mldsa87"	ML-DSA (자동)	mldsa87 (ML-DSA만 신뢰)

1단계 OTA가 모든 대상 디바이스에 끝난 뒤 3단계 .swu를 배포한다. 1단계를 받지 못한 디바이스는 3단계 .swu를 거부한다.

5.3 ML-DSA-87 → RSA-3072 (되돌리기)

단계	local.conf	.swu 서명	설치 후 디바이스
1	SWU_VERIFY_MODE = "dual", SWU_SIGN_ALG = "mldsa87"	ML-DSA	dual
2	SWU_VERIFY_MODE = "rsa3072"	RSA (자동)	rsa3072 (기존 FIPS 설정으로 돌아감)

5.4 dual 상태에서 서명만 바꾸기

dual 장비에는 SWU_SIGN_ALG만 바꿔 빌드한 .swu를 그대로 보낼 수 있다. 디바이스 쪽 변화는 없다.

5.5 플래시로 설치하는 경우

STM32CubeProgrammer로 이미지를 직접 쓰는 공장 설치에는 이전 이미지가 없으므로 순서 제약이 없다. 원하는 모드로 빌드한 이미지를 바로 쓴다. 이후 OTA부터 5.1절 규칙을 따른다.

5.6 잘못된 순서로 보냈을 때

디바이스는 설치하지 않고 현재 이미지를 유지한다. swupdate 로그:

```
START Software Update started !
FAILURE ERROR : Signature verification failed
FAILURE ERROR : Compatible SW not found
FATAL_FAILURE Image invalid or corrupted. Not installing ...
```

ustate=0, swupdate 서비스는 계속 동작한다. 디바이스가 신뢰하는 알고리즘으로 다시 서명해 보내면 된다 (dual이면 SWU_SIGN_ALG만, 단일 모드이면 그 모드에 맞춰 다시 빌드).

6. 디바이스 확인

디바이스에서 root로 실행한다.

```
# 신뢰 인증서 전체 (x509 명령은 첫 인증서만 보여 줌). rsa3072 장비에는 swupdate.cnf가 없음
C=/etc/ssl/swupdate.cnf; [ -f $C ] || C=/etc/ssl/openssl.cnf
OPENSSL_CONF=$C openssl storeutl -noout -certs -text /usr/share/swupdate/swu-sign-cert.pem \
```

```
| grep -E '^[0-9]+: |Subject:|Public Key Algorithm|Total found'
```

```
# dual / mldsa87이면 둘 다 있어야 함 (rsa3072이면 없음)
```

```
ls -l /etc/ssl/swupdate.cnf /etc/swupdate/conf.d/10-openssl
```

```
# swupdate 데몬이 swupdate.cnf를 받았는지
```

```
for P in $(pidof swupdate); do tr '\0' '\n' < /proc/$P/environ | grep OPENSSL_CONF; done
```

dual·mldsa87 장비에서 OPENSSL_CONF=/etc/ssl/swupdate.cnf 없이 인증서를 출력하면 ML-DSA 인증서에서 X509_PUBKEY_get0:decode error가 나온다. 기본 설정(FIPS 전용)에 ML-DSA가 없어서이며 정상이다.

7. 주의사항

- **범위:** OTA 패키지 서명만 바뀐다. 부트체인(TF-A/FIP, FIT 이미지 서명 RSA-2048, ROM ECDSA)과 .swu 내부 페이로드 해시(SHA-256)는 그대로이다.
- **FIPS 경계:** dual·mldsa87에서는 swupdate의 암호 연산(RSA 서명 검증 포함)이 default provider를 쓸 수 있어 FIPS 경계 밖이다. rsa3072만 기존 FIPS 설정을 유지한다.
- **이중 서명은 쓰지 않는다:** 한 .swu에 RSA와 ML-DSA 서명을 함께 넣으면, swupdate가 모든 서명자를 검증하므로 ML-DSA를 모르는 디바이스는 거부한다. 신뢰 인증서를 두 개 두는(dual) 방식으로 전환한다.
- **키 교체:** 같은 알고리즘의 키를 바꾸는 경로는 아직 없다. 디렉터리마다 인증서가 하나라서, 키를 바꾸면 기존 디바이스가 새 .swu를 거부한다.
- **개발 키:** orb의 ~/STM32AP_KeyGen 키(RSA CN=shiba SWU Key, ML-DSA CN=shiba SWU Key ML-DSA-87)는 개발용이다. 출하용 키는 서명 전용 머신에서 만들어 /opt/STM32AP_KeyGen에 보관하고 빌드 머신에 같은 파일을 배포한다.